BOLSA DE TRABAJO · MVP

Arquitectura del sistema

Vista general, arquitectura por capas, flujo de trabajo y tecnologías utilizadas, con el motivo de cada una.

TP2 · Elementos de Programación IA y LowCode · 2.º cuatrimestre 2026 · Grupo 1
Proyecto A: Plataforma de vinculación y desarrollo profesional · Noviembre de 2026

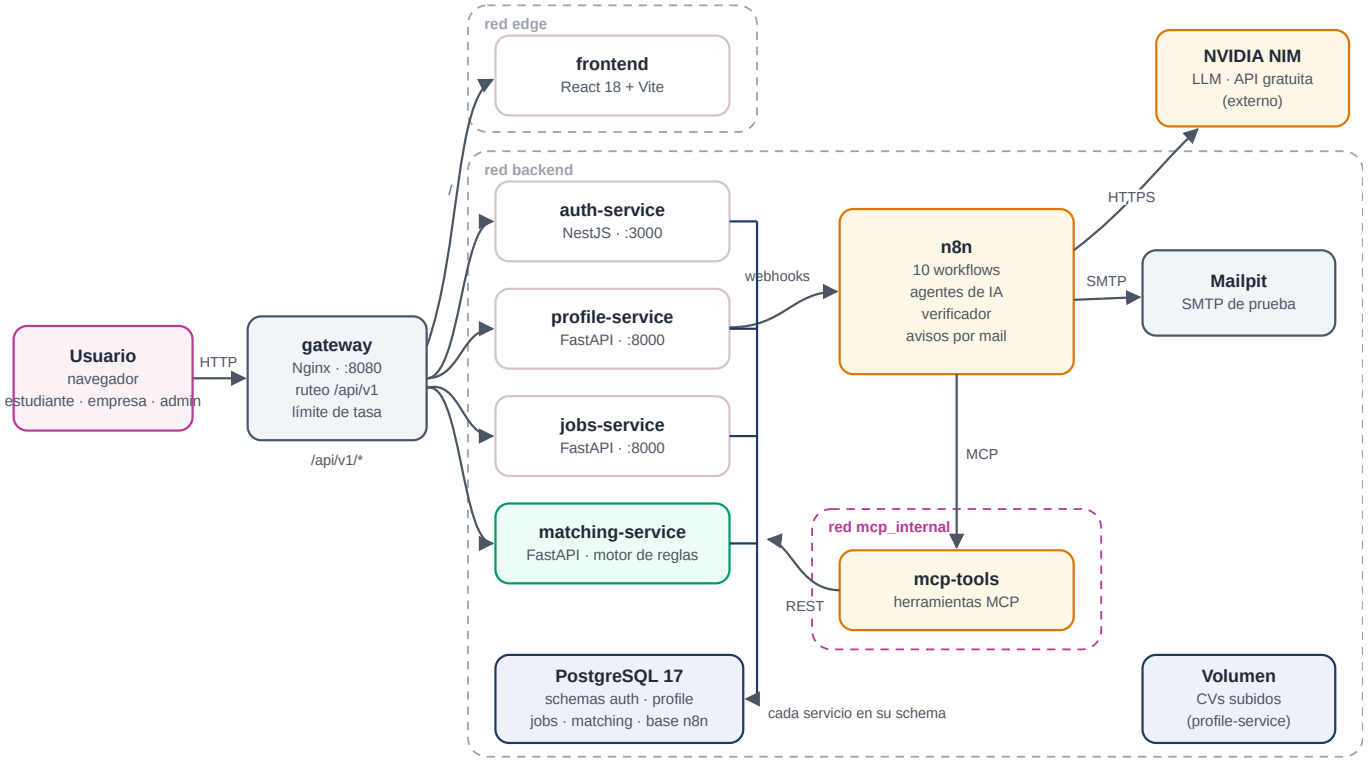
Contenido

1. Vista general
2. Arquitectura por capas
3. Flujo de trabajo
4. Tecnologías y por qué se usan
5. Decisiones clave

1. Vista general

La Bolsa de Trabajo es un sistema de **microservicios REST** detrás de un API Gateway. La lógica de negocio está repartida en cuatro servicios, cada uno con su propio schema de base de datos. La IA y las automatizaciones viven en **n8n**. Todo corre con Docker Compose, salvo los modelos de lenguaje, que se consumen desde la API gratuita de NVIDIA NIM.

Reglas (determinístico) · IA (sugerencias verificadas) · Infraestructura · Datos · Usuario

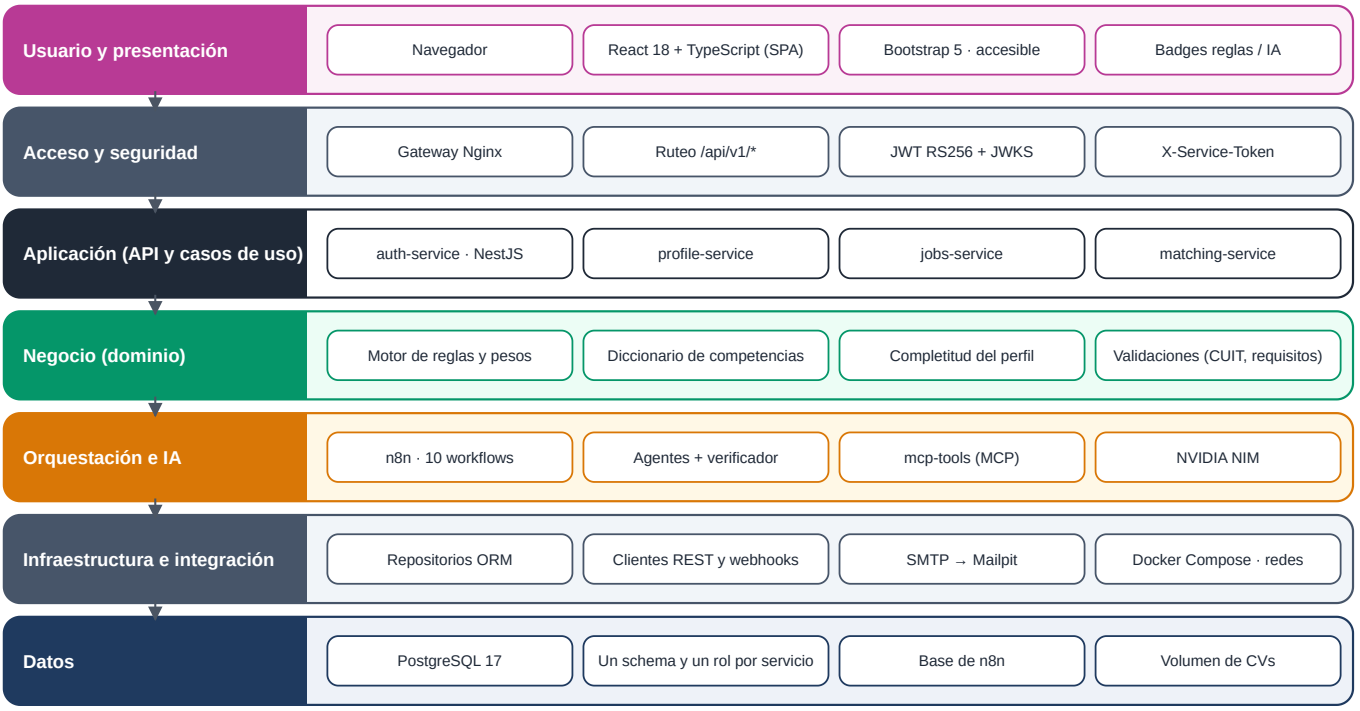


Componentes, redes y comunicación. La barra azul a la derecha de los servicios es la persistencia: cada servicio escribe sólo en su schema de PostgreSQL.

- **Una sola entrada:** el usuario sólo habla con el gateway (`:8080`), que sirve el front y deriva `/api/v1/*`.
- **Redes separadas:** `edge` para el front, `backend` para los servicios y `mcp_internal` como canal exclusivo n8n ↔ mcp-tools, que no pasa por el gateway.
- **Eventos por webhook:** los servicios avisan a n8n (CV subido, oferta nueva, perfil actualizado, pedido de recomendaciones) y n8n responde por REST.

2. Arquitectura por capas

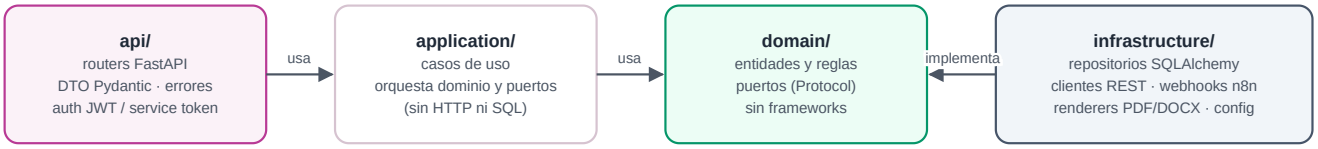
Vista de arriba hacia abajo: cada capa usa sólo a la de abajo. El color de cada tarjeta indica si el resultado es **por reglas** (verde, siempre igual y auditable) o **de IA** (ámbar, sugerencias verificadas).



Capas del sistema, de la interfaz a los datos.

Capa	Responsabilidad
Usuario y presentación	Pantallas de estudiante, empresa y admin. Muestra separados el puntaje por reglas (badge verde) y la sugerencia de IA (badge ámbar, con aviso).
Acceso y seguridad	Entrada única, ruteo, límite de tasa y autenticación: usuarios con JWT, servicios con token propio.
Aplicación	APIs REST versionadas (`/api/v1`) y casos de uso: registrar, guardar el perfil, publicar una búsqueda, pedir recomendaciones.
Negocio	Reglas puras y testeadas: puntaje de compatibilidad, competencias faltantes, completitud del perfil y validaciones.
Orquestación e IA	Workflows de n8n, agentes y verificador. La IA propone; el usuario y el verificador controlan.
Infraestructura	Adaptadores a la base, a otros servicios, al correo y al entorno de contenedores.
Datos	PostgreSQL con un schema y un rol por servicio, la base de n8n y el volumen de CVs.

Capas dentro de cada servicio

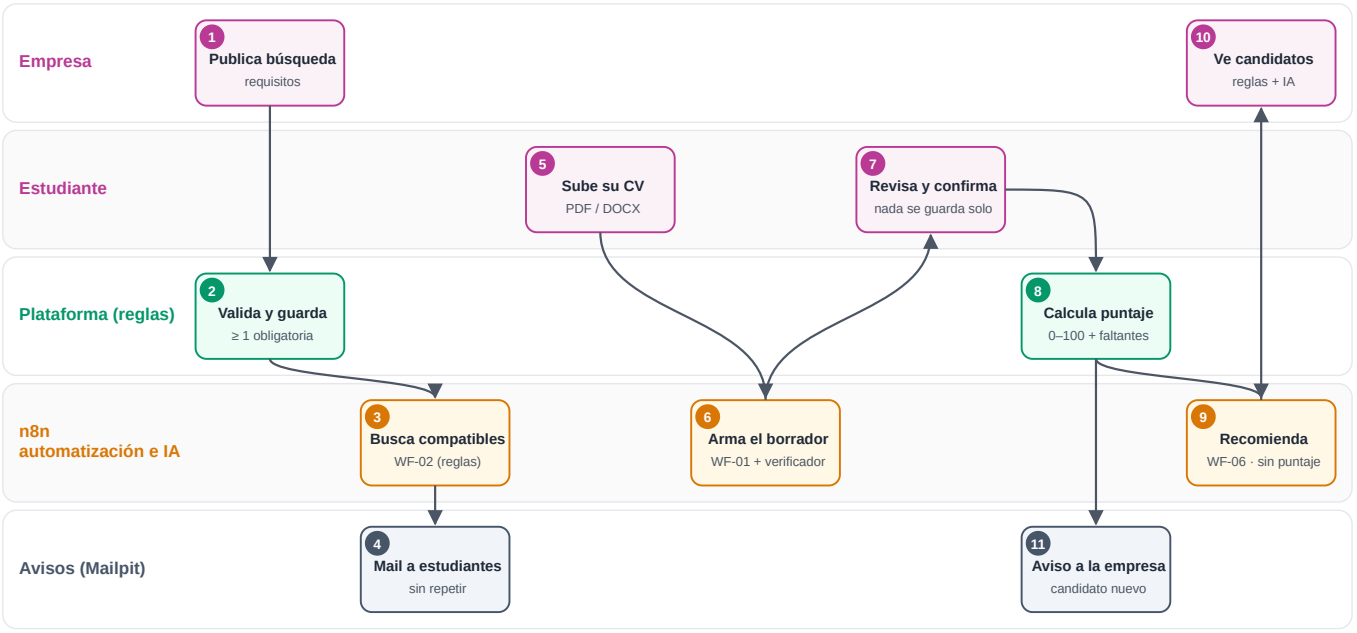


Las flechas apuntan hacia el dominio: infraestructura implementa sus puertos (inversión de dependencias).
auth-service sigue la misma idea en NestJS: controllers delgados → services → dominio ← repositorios TypeORM.

Estructura interna de profile-, jobs- y matching-service.

3. Flujo de trabajo

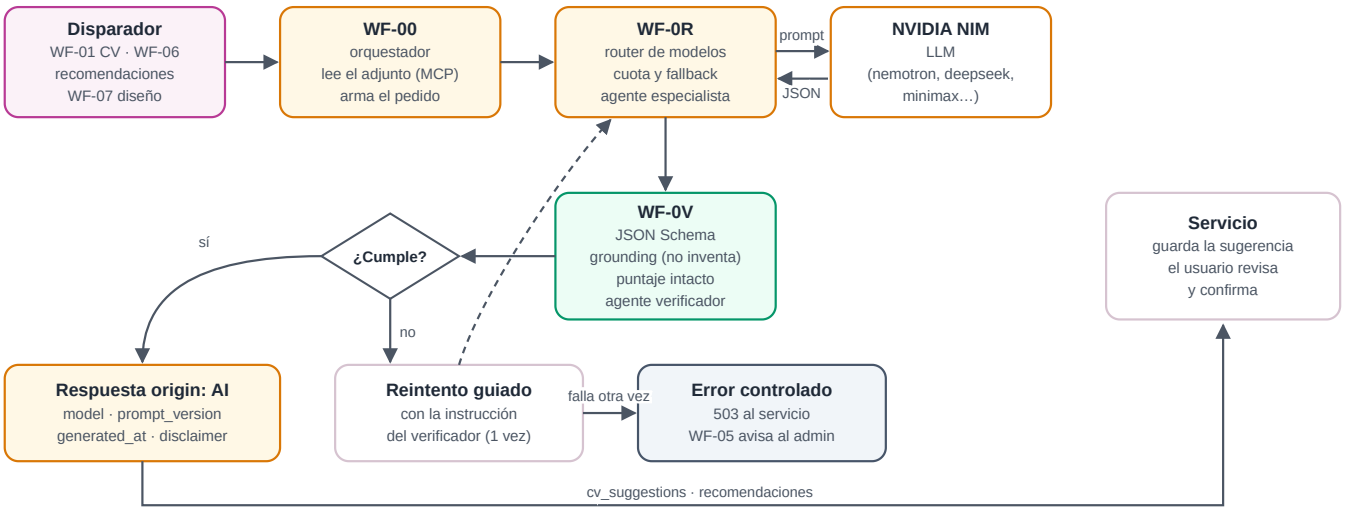
El circuito completo, desde que una empresa publica una búsqueda hasta que ve los candidatos, pasando por el estudiante que sube su CV.



Flujo del negocio por actor. Los pasos verdes los resuelve el motor de reglas; los ámbar, la IA.

1. La **empresa** publica una búsqueda; jobs-service exige al menos una competencia obligatoria.
2. n8n (WF-02) busca estudiantes compatibles por reglas y les avisa por mail, sin repetir avisos.
3. El **estudiante** sube su CV; la IA arma un borrador, el verificador lo controla y el estudiante revisa y confirma. Nada se guarda sin su confirmación.
4. matching-service calcula el puntaje (0–100), el desglose y las competencias faltantes.
5. La IA suma recomendaciones (por qué conviene, qué tener en cuenta, qué estudiar) sin tocar el puntaje.
6. La empresa ve los candidatos ordenados por reglas, con el análisis de la IA, y recibe un aviso cuando aparece uno nuevo.

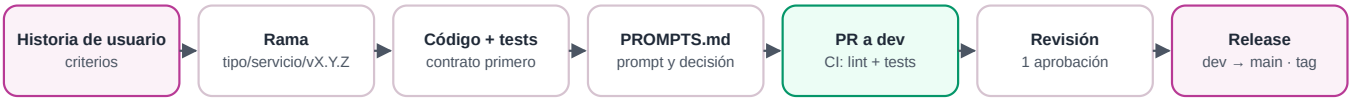
Flujo de los agentes de IA



Cada pedido de IA pasa por el orquestador, el router de modelos y el verificador.

> La IA **nunca calcula ni modifica el puntaje**: lo recibe del motor y el verificador rechaza cualquier respuesta que lo cambie. Toda salida de IA lleva `origin: "AI"`, el modelo, la versión del prompt, la fecha y un aviso.

Flujo de desarrollo



Cada cambio va en su rama, con tests, contrato y registro del prompt, y entra a dev por PR con CI.

4. Tecnologías y por qué se usan

Usuario y presentación

Tecnología	Dónde	Por qué se usa
React 18 + TypeScript	frontend	Interfaz por componentes reutilizables (perfil, ofertas, puntaje, IA). TypeScript detecta errores de tipos contra los contratos de la API antes de ejecutar.
Vite	frontend	Servidor de desarrollo con recarga instantánea y build optimizado para producción.
Bootstrap 5 + Bootstrap Icons	frontend	Componentes accesibles y responsive listos para usar: el equipo se concentró en el negocio y no en CSS. Contraste AA y foco visible.
Vitest	frontend	Tests unitarios del front con la misma configuración de Vite.

Acceso y seguridad

Tecnología	Dónde	Por qué se usa
Nginx (nginx-unprivileged 1.27)	gateway	Único punto de entrada: sirve el front y deriva /api/v1/* a cada servicio. Al ser mismo origen no hace falta CORS. Limita la tasa de inicios de sesión y corre sin root.
JWT RS256 + JWKS	auth y servicios	Sólo auth-service firma con la clave privada; los demás validan el token localmente con la pública, sin llamar a auth en cada pedido.
argon2	auth-service	Hash de contraseñas recomendado por OWASP: resistente a ataques por GPU.
X-Service-Token	entre servicios	Autentica las llamadas internas y los webhooks de n8n, separadas de las de usuarios.

Aplicación y negocio

Tecnología	Dónde	Por qué se usa
FastAPI + Pydantic + Uvicorn	profile, jobs, matching	Python, como pide la consigna. Valida la entrada con tipos, devuelve errores HTTP claros y genera la documentación OpenAPI (Swagger y ReDoc) sola.
NestJS 11 + TypeORM + Passport	auth-service	Framework con inyección de dependencias y módulos de seguridad maduros (JWT, estrategias). class-validator valida los DTO y @nestjs/swagger documenta la API.
Arquitectura por capas + SOLID	todos los servicios	El dominio (motor de reglas, completitud, validaciones) no depende de frameworks ni de la base: se prueba con tests unitarios y se cambia sin tocar la API.

PyYAML (scoring_rules.yaml)	matching-service	Los pesos del puntaje se ajustan en un archivo, sin cambiar código.
WeasyPrint + Jinja2 + python-docx	profile-service	Generan el CV nuevo en PDF, HTML y Word desde una misma plantilla, con texto real para los sistemas de selección.

Orquestación e IA

Tecnología	Dónde	Por qué se usa
n8n 2.45.7	n8n	Orquestación low-code pedida por la consigna: webhooks, condiciones, mails y nodos nativos de agentes de IA (LangChain), con cada ejecución visible para auditar.
NVIDIA NIM	externo	Modelos de lenguaje gratuitos con API compatible con OpenAI. El router reparte la cuota por minuto y cambia de modelo si uno falla.
MCP (SDK oficial, FastMCP)	mcp-tools	Protocolo estándar para darles herramientas a los agentes (leer el CV, rankings, validar schema, grounding). Cada agente ve sólo las herramientas que usa.
jsonschema + pypdf + python-docx	mcp-tools	Validan la salida de la IA contra el contrato y extraen el texto de los CV en PDF y Word.
Verificador (WF-0V)	n8n + mcp-tools	Controla schema, que la IA no invente competencias y que el puntaje sea el del motor. Si falla, reintenta una vez con instrucciones.

Infraestructura y datos

Tecnología	Dónde	Por qué se usa
PostgreSQL 17	postgres	Base relacional robusta con soporte JSON. Un schema y un rol por servicio: ningún servicio puede leer los datos de otro.
SQLAlchemy 2 + psycopg 3	servicios Python	Repositorios que separan el dominio de la base; los tests usan repositorios en memoria.
Docker + Docker Compose	todo el sistema	Todo se levanta con un comando, igual en cualquier máquina. Redes separadas (edge, backend, mcp_internal), healthchecks, usuarios sin root y volúmenes externos.
Mailpit	mail	Servidor SMTP de prueba: se ven los mails sin enviarlos a casillas reales.
GitHub Actions	CI	En cada PR corre lint y tests sólo de los servicios que cambiaron.
ruff, black, mypy, pytest · Prettier, tsc, Jest	calidad	Estilo y tipos uniformes y tests unitarios, con cobertura mínima del 70 % en dominio y aplicación.

5. Decisiones clave

Decisión	Motivo
Reglas separadas de la IA	El puntaje tiene que ser explicable y auditable: lo calcula siempre el motor, con pesos visibles. La IA sólo agrega contexto y consejos.
Sin servicio intermedio de IA	Toda la IA vive en n8n: se aprovechan sus nodos de agentes y de MCP, y cada ejecución queda visible para revisarla.
Supervisión humana	Lo que propone la IA (datos del CV, título, diseño) se precarga como sugerencia y el usuario lo confirma.
Un schema y un rol por servicio	Ningún servicio accede a los datos de otro; se comunican por REST y eventos.
Contratos primero	Cada servicio tiene su contrato en `contracts/` y su OpenAPI vivo; un cambio de API actualiza el contrato en el mismo PR.
IA gratuita con respaldo	NVIDIA NIM sin costo; el router reparte la cuota y cambia de modelo si uno falla o se demora.
Todo en contenedores	Se instala igual en cualquier máquina; los datos viven en volúmenes externos que sobreviven a recrear los contenedores.

Más detalle en `docs/ARQUITECTURA.md` (diagramas Mermaid, puertos y recorridos de datos), `docs/BITACORA_IA.md` y `contracts/`.